

the basic skeleton for a Django project. Figure 1 shows the directory skeleton structure.

The `__init__.py` file in most of the directories of the project. Its purpose is to mark the directories on a package directories, to be imported by Python.

For example, if we have a directory `foo/` and it contains the files `foo.py` and `linux.py`, we can access its contents from `foo.linux` as follows:

```
#!/usr/bin/env python
```

The `manage.py` file is the project management file. It lets us do further management tasks on the current project after the basic skeleton is in place. We will use `manage.py` to start the development server, to create new applications, to synchronise database tables, and other things inside the project directory in which `manage.py` exists.

The `settings.py` file holds the different settings variables for the Django project. It includes the time zone used, database engine and connectivity, templates, additional authentication support, etc. It manages the overall settings for the project.

The `urls.py` file is the URL map file for the current Django project. In `urls.py`, we will add different URL mapping schemes. For example, if we have some URL, the function of this file is to specify what to do when that URL is displayed as the output. It has a pretty good regex engine to handle any customisation options. It directs each URL to a view function, which returns the output HTTP response. If a non-existent URL is requested, we can direct it to a 404 Error page. Or if we request a URL in the form `http://example.com/page/(digits)/` (for example, `http://example.com/page/11/`), we can write custom regular expressions such that this Web page will always be handled by a custom-written page function and the `(digits)` is the parameter.

Let's look at how to set up a basic Django application.

First, we will use the Django development server to test our application. After development, we will deploy it on an Apache server with Django extension support. Then, we will move the project skeleton directory that we have created to the production location and use the following command:

```
python manage.py runserver
```

```
1.1, using settings 'helloworld.settings'
r is running at http://127.0.0.1:8000/
Hit CTRL-C to stop the server
```

Figure 1: Directory structure of the 'helloworld' Django project

The above snippet informs us that a development server has started and we can access the output pages from the local host machine by pointing a Web browser to the following URL: `http://127.0.0.1:8000/`. By default, the development server runs at port 8000. We can change it by using the desired port number as an additional argument along with the `manage.py` command as follows:

```
python manage.py runserver port_number
```

Shell

There is an option provided by `manage.py`—the Django *shell*. We can use it to access the Django Python interpreter (which runs the current project's Python interpreter with its environment set-up). It is very helpful during the debugging process:

```
python manage.py shell
Python 2.6.4rc2 (r264rc2:75487, Oct 20 2009, 02:55:11)
[GCC 4.4.1] on linux2
Type "help()", "copyright()", "credits()" or "license()" for more information.
(InteractiveConsole)
>>>
```

We need to write a sample Web page using Django now. Create a file `views.py` and append the following lines in it:

```
from django.http import HttpResponseRedirect
def index(request):
    html = "<h1>Hello Web !</h1>"
    return HttpResponseRedirect(html)
```

The above snippet of code is known as a 'view'. Views generate the HTML data for the Web page.

We need to set up the `urls.py` next. Add the following line to `urls.py`:

```
(r'^helloworld/$', 'helloworld.views.index'),
```

...so that it looks like the following code:

```
from django.conf.urls.defaults import *
urlpatterns = patterns('',
    (r'^helloworld/$', 'helloworld.views.index'),
)
```

Now, using a browser, check out the following Web page: `http://localhost:8000/helloworld`. You can see the output returned by the `index()` function.